BEGINNER FRIENDLY · ZERO TO BASICS

Java Basics, Explained From Zero

Real-life analogies · Diagrams · Runnable examples

01
Java Fundamentals & Program Structure

02
Variables, Types & Operators

03
Control Statements & Loops

```
class Hello {
  public static void main(String[] args) {
    System.out.println("Welcome to Java");
  }
}
```

A Beginner's Guide to Java Programming

Table of Contents

PART ONE

Java Fundamentals

1 What is Java?

REAL-LIFE ANALOGY

Think about a restaurant. You tell the waiter **"Give me one dosa."** The waiter takes your instruction to the kitchen. Java works the same way — you give the computer an instruction, and it carries it out.

```
System.out.println("Give me one dosa");
```

DEFINITION

Java = A language used to give instructions to a computer.

2 Basic Idea of a Java Program

REAL-LIFE ANALOGY

A school has classrooms, classrooms have students, and students perform activities. A Java program follows the same nested structure.

```
Java Program
     ↓
   Class
     ↓
 main() method
     ↓
  Statements
     ↓
Computer performs actions
```

```java
class Hello {
    public static void main(String[] args) {

        System.out.println("Hello");

    }
}
```

```
class Hello
     ↓
Our program/container
```

```
main()
     ↓
Starting point
```

3 Writing a Simple Java Program

Let's make the computer say **Welcome to Java**:

```java
class Welcome {
    public static void main(String[] args) {

        System.out.println("Welcome to Java");

    }
}
```

OUTPUT

```
Welcome to Java
```

REAL-LIFE ANALOGY

Imagine a robot: **START** → **Say "Welcome to Java"** → **STOP**. The `main()` method is where a Java program starts executing.

4 What is a Class?

REAL-LIFE ANALOGY

A class is like a house blueprint — it describes what rooms, doors, and windows the house will have, without being the house itself.

```
class Student {

    String name;
    int age;

}
```

```
Student
  ├── name
  └── age
```

5 What is main()?

REAL-LIFE ANALOGY

Before a movie starts, there's a clear starting point. Java needs one too — that's `public static void main(String[] args)`.

```
class Demo {

    public static void main(String[] args) {

        System.out.println("Program started");

    }
}
```

```
Java starts → Finds main() → Executes statements → Program finishes
```

6 Java Statements

A statement is simply an instruction given to the computer. Every statement usually ends with a semicolon **;** — just like a full stop in English.

```
int age = 25;
System.out.println(age);
```

REMEMBER

";" in Java = "." at the end of an English sentence.

7 Elements / Tokens in Java

Java breaks every line into small pieces called **tokens** — just like an English sentence is made of individual words.

```
int     age     =     25     ;
 ↓       ↓      ↓     ↓      ↓
keyword identifier operator literal separator
```

Keywords

Reserved words already used by Java — you can't use them as variable names.

`class` `int` `if` `else` `for` `while` `public` `static` `void`

Identifiers

A name you give to something — a variable, class, or method.

```java
int age = 25;         // age is an identifier
String studentName = "Ravi";  // studentName is an identifier
```

Literals

A fixed value written directly in the program.

`10` `25` `3.14` `"Hello"` `'A'` `true` `false`

Operators

Symbols that perform operations, such as `+ - * / = > < ==`

```java
int salary = 30000;
int bonus = 5000;

int total = salary + bonus;
```

```
OUTPUT
35000
```

8 Program Structure

```java
class Employee {

    public static void main(String[] args) {

        int salary = 30000;

        System.out.println(salary);

    }
}
```

```
Class
 ├── main()
      ├── variable
      ├── calculation
      └── output
```

```
Company
   ↓
Department
   ↓
 Employee
   ↓
   Work
```

9 Command Line Arguments

```
class Welcome {

    public static void main(String[] args) {

        System.out.println(args[0]);

    }
}
```

Run: `java Welcome Ravi`

```
OUTPUT
Ravi
```

> **REAL-LIFE ANALOGY**
> Ordering food: you say "Order: Pizza" and the restaurant receives that input directly. Similarly `Ravi` is passed into the program through `String[] args`.

Multiple Arguments

```
class Student {

    public static void main(String[] args) {

        System.out.println(args[0]);
        System.out.println(args[1]);

    }
}
```

Run: `java Student Ravi 25`

```
OUTPUT
Ravi
25
```

10 User Input

Another way to get input — while the program is running — is using **Scanner**.

```java
import java.util.Scanner;

class Student {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter your name: ");

        String name = sc.nextLine();

        System.out.println("Hello " + name);
    }
}
```

```
SAMPLE RUN
Enter your name: Ravi
Hello Ravi
```

11 Why Do We Need User Input?

```
Program → Ask user → Receive input → Process input → Give output
```

REAL-WORLD USES

ATM (PIN, amount) · Banking (account number, amount) · Online exams (name, marks)

12 Different Types of User Input

Type	Code	Example Input
String	`sc.nextLine()`	Ravi Kumar
Integer	`sc.nextInt()`	25
Double	`sc.nextDouble()`	45000.50
Character	`sc.next().charAt(0)`	M

Real-Time Example

13 Simple ATM Program

```java
import java.util.Scanner;

class ATM {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter balance: ");
        double balance = sc.nextDouble();

        System.out.print("Enter withdrawal amount: ");
        double amount = sc.nextDouble();

        double remaining = balance - amount;

        System.out.println("Remaining balance: " + remaining);
    }
}
```

```
SAMPLE RUN
Enter balance: 10000
Enter withdrawal amount: 2500
Remaining balance: 7500
```

User → Input → Java program → Calculation → Output

14 Escape Sequences

Escape	Meaning	Example
\n	New line	Hello\nWorld
\t	Tab space	Name:\tRavi
\"	Double quote	\"Hello\"
\'	Single quote	\'A\'
\\	Backslash	C:\\Java

```java
System.out.println("Name:\tRavi");
System.out.println("Age:\t25");
```

```
OUTPUT
Name:    Ravi
Age:     25
```

15 Comments

A comment is a note for humans — Java ignores it completely.

Single-line

```java
// Calculate total salary
int total = salary + bonus;
```

Multi-line

```java
/*
 Employee Salary
 Program
*/
```

16 Programming Style

✘ Bad Style

```java
class Student{public static void
main(String[]args){int
age=20;System.out.println(age);}}
```

✓ Good Style

```java
class Student {

    public static void main(String[] args) {

        int age = 20;
        System.out.println(age);
    }
}
```

17 Naming Style

✘ Bad

```java
int x = 50000;
```

✓ Better

```java
int salary = 50000;
```

Real-Time Example

18 Employee Salary Program

```java
import java.util.Scanner;

class EmployeeSalary {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter employee name: ");
        String employeeName = sc.nextLine();

        System.out.print("Enter basic salary: ");
        double basicSalary = sc.nextDouble();

        System.out.print("Enter bonus: ");
        double bonus = sc.nextDouble();

        double totalSalary = basicSalary + bonus;

        System.out.println("Total Salary: " + totalSalary);
    }
}
```

```
SAMPLE RUN
Enter employee name: Ravi
Enter basic salary: 30000
Enter bonus: 5000
Total Salary: 35000.0
```

Class → main() → Statements → Variables → User Input → Operators → Output → Comments → Escape sequence

PART TWO

Variables to Operators

1 Variables

REAL-LIFE ANALOGY

A variable is a named box that stores a value — like labeled boxes in a college office.

```
int studentAge = 20;
String studentName = "Ravi";
double marks = 85.5;
```

2 Data Types

Type	Meaning	Example
int	Whole number	25
double	Decimal	25.5
String	Text	"Ravi"
char	One character	'A'
boolean	True / False	true
long	Very large whole number	9000000000L

3 Type Casting

```
double price = 99.99;
int amount = (int) price;  // 99.99 -> 99 (narrowing)
```

```
int marks = 85;
double result = marks;  // 85 -> 85.0 (widening)
```

REAL-TIME USE

```
double percentage = (double) obtainedMarks / totalMarks * 100;
```

4 Scope of a Variable


```java
if (true) {
    int marks = 90;
}
System.out.println(marks); // ERROR - out of scope
```

5 Identifier

✓ Valid

studentAge, student1, totalMarks

✘ Invalid

1student, student-name, class

6 Literal Constants

`25` `85.5` `'A'` `"Ravi"` `true`

7 Static Variables

REAL-LIFE ANALOGY

Every student in a college shares the same college name — instead of storing a separate copy for each student, we store it once as `static`.

```java
class Student {

    static String college = "ABC Engineering College";
    String name;

}
```

8 final Attribute

```
final int MAX_MARKS = 100;
System.out.println(MAX_MARKS);  // OK
MAX_MARKS = 200;                // ERROR
```

COMMON USES

`final double PI = 3.14159;` · `final int DAYS_IN_WEEK = 7;`

9 Introduction to Operators

`+` `-` `*` `/` `%` `>` `<` `=`

10 Assignment Operator =

```
int age = 25;   // store 25 inside age
```

IMPORTANT

= means "put a value" — not "is equal to". For comparison, Java uses ==.

11 Basic Arithmetic Operators

Operator	Meaning	Example	Result
+	Addition	10 + 5	15
-	Subtraction	10 - 5	5
*	Multiplication	10 * 5	50
/	Division	10 / 5	2
%	Remainder	10 % 3	1

MODULUS REAL-TIME USE

Checking even/odd: `if (number % 2 == 0) { "Even" }`

12 Increment ++

```
int visitors = 100;
visitors++;   // visitors = 101
```

13 Decrement --

```java
int stock = 10;
stock--;   // stock = 9, after one item is sold
```

14 Pre-Increment vs Post-Increment

Post-increment: x++

```java
int x = 5;
int y = x++;
// y = 5, then x = 6
```

use first, increase later

Pre-increment: ++x

```java
int x = 5;
int y = ++x;
// x = 6, then y = 6
```

increase first, use later

15 Ternary Operator ?:

```java
int marks = 80;
String result = (marks >= 40) ? "Pass" : "Fail";
```

16 Relational Operators

Operator	Meaning	Example
>	greater than	10 > 5
<	less than	10 < 5
>=	greater than or equal	10 >= 10
<=	less than or equal	5 <= 10
==	equal to	10 == 10
!=	not equal	10 != 5

17 = vs ==

= (Assignment)

```java
age = 25;
```

PUT a value

== (Comparison)

```java
age == 25
```

CHECK a value → true / false

PART THREE

Java Control Statements

1 if Statement

```java
int marks = 75;

if (marks >= 40) {
    System.out.println("Pass");
}
```

OUTPUT

```
Pass
```

2 Nested if

```java
boolean isStudent = true;
boolean hasHallTicket = true;

if (isStudent) {
    if (hasHallTicket) {
        System.out.println("You can enter the exam hall");
    }
}
```

```
Is student? → YES → Has hall ticket? → YES → Enter exam hall
```

3 if-else & Multiple if-else

```java
int marks = 85;

if (marks >= 90) {
    System.out.println("A+");
} else if (marks >= 80) {
    System.out.println("A");
} else if (marks >= 70) {
    System.out.println("B");
} else {
    System.out.println("Fail");
}
```

NOTE

Only the first matching condition executes.

4 switch Statement

```java
int choice = 2;

switch (choice) {
    case 1: System.out.println("Add"); break;
    case 2: System.out.println("Delete"); break;
    case 3: System.out.println("Search"); break;
    default: System.out.println("Invalid choice");
}
```

OUTPUT

```
Delete
```

WHY BREAK?

Without `break`, execution falls through into the next cases.

5 while & for Loops

while

```
int i = 1;
while (i <= 5) {
    System.out.println(i);
    i++;
}
```

for

```
for (int i = 1; i <= 5; i++) {
    System.out.println(i);
}
```

OUTPUT (BOTH)

```
1
2
3
4
5
```

RULE OF THUMB

Known number of repetitions → `for`. Condition-based repetition → `while`.

6 break & continue

break — stop the loop

```
for (int i=1;i<=10;i++){
  if (i==5) break;
  print(i);
}
```

Output: 1 2 3 4

continue — skip this round

```
for (int i=1;i<=5;i++){
  if (i==3) continue;
  print(i);
}
```

Output: 1 2 4 5

Real-Time Example

7 ATM PIN Attempts

```
int correctPin = 1234;

for (int attempt = 1; attempt <= 3; attempt++) {

    int enteredPin = 1234;

    if (enteredPin == correctPin) {
        System.out.println("Login successful");
        break;
    } else {
        System.out.println("Wrong PIN");
    }
}
```

```
for → repeat max 3 times
if / == → compare PIN
break → stop once correct
```